LeMauvaisCoin

Présentation globale - SQL

Maxime Richard / Alexandre MULARD

Sommaire

- Modélisation et Schéma
- Requêtes complexes et Transactions
- Performance et Optimisation
- Présentation du site
- Conclusion

Modélisation et Schéma

Relation de la base de donnée :

1-N	N-N
category ↔ product	user ↔ type
user ↔ sale_ad	command ↔ sale_ad
product ↔ sale_ad	
user ↔ command	
sale_ad ↔ picture	

Modélisation et Schéma

Modélisation et Schéma

- Entitées : User / Type / User_Type / Category / Product / Sale_id / Picture / Command / Command_product
- Normalisation (3NF) :
  - 1 seule valeur
  - 1 seul concept
  - Ne pas dépendre d’une autre colonne qui n’est pas la clé

Requêtes complexes et Transactions

Requête complexe (Produit le plus vendu) :

```
SELECT
    p.name AS nom_produit,
    cat.name AS categorie,
    COUNT(cp.sale_ad_id) AS nombre_de_ventes
FROM product p
JOIN category cat ON p.category_id = cat.id
JOIN sale_ad sa ON p.id = sa.product_id
JOIN command_product cp ON sa.id = cp.sale_ad_id
GROUP BY p.id, p.name, cat.name
ORDER BY nombre_de_ventes DESC
LIMIT 1;
```

Requêtes complexes et Transactions

Transaction :

```
BEGIN;

INSERT INTO command (date, user_id)

VALUES (CURRENT_TIMESTAMP, 3);

INSERT INTO command_product (command_id, sale_ad_id)

VALUES (LASTVAL(), 3);

UPDATE sale_ad

SET quantity = quantity - 1

WHERE id = 3;
```

Si tout est OK :

```
COMMIT;
```

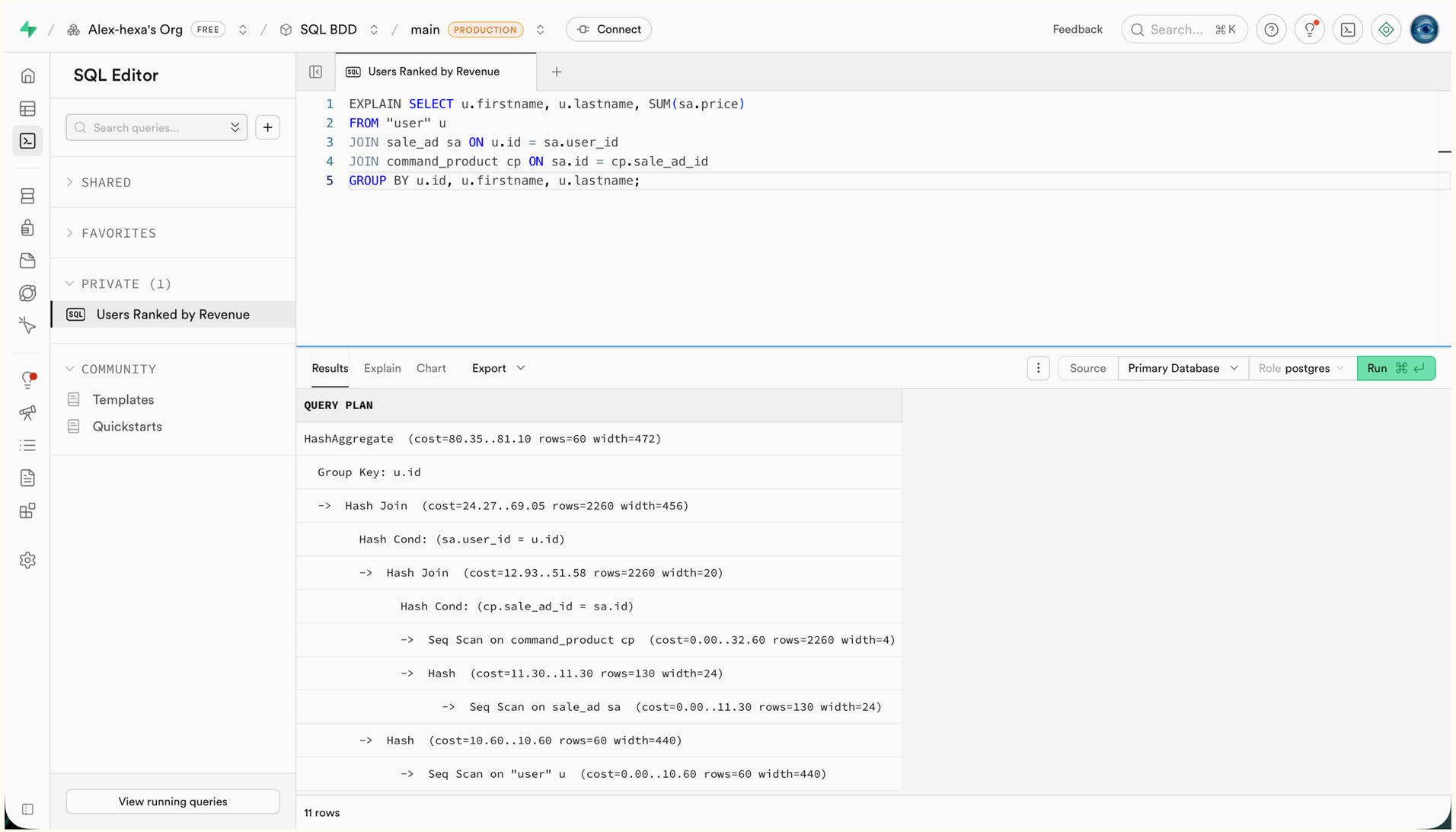
Si une erreur survient (ex: stock < 0), Postgres permet de faire :

```
ROLLBACK;
```

Performance et Optimisation

- Diagnostic : CA - EXPLAIN QUERY PLAN
- Optimisation : INDEX sur sale_id
- Recherche textuelle : INDEX GIN (mot-clé)

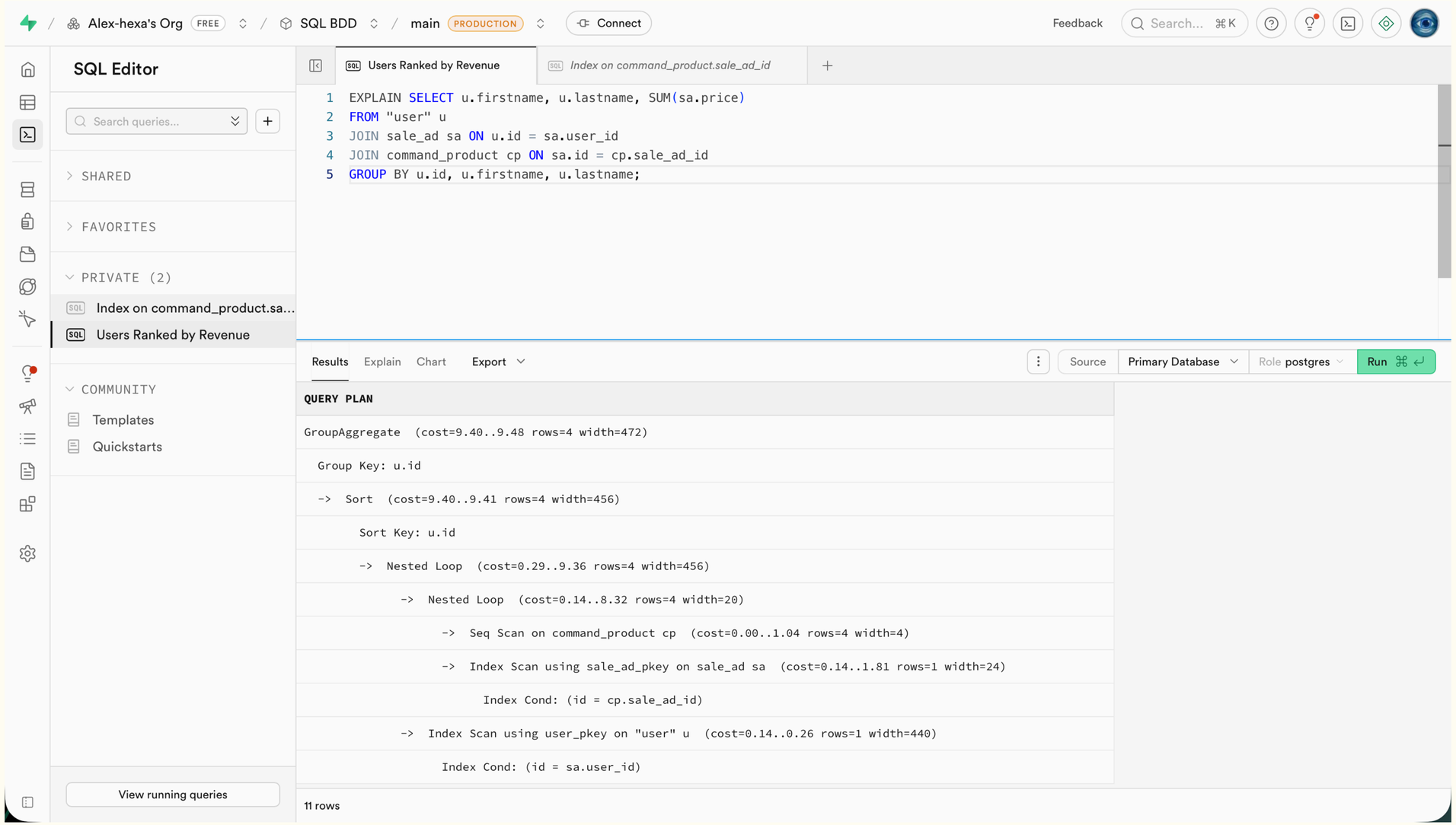
Performance et Optimisation



Performance et Optimisation

- CREATE INDEX idx_cp_sale_ad ON command_product(sale_ad_id);

Performance et Optimisation



Présentation

Conclusion

Merci pour votre écoute

Avez-vous des questions ?